EBIS Business Techschool

Tema 4 · Clase 2

Prompt engineering para automatización y salidas estructuradas

De respuestas naturales a objetos de negocio parseables, validables y seguros.

Máster en Ingeniería de Automatización con IA Agéntica



Clase 2

Seguimos construyendo la aplicación LLM: de invocar el modelo a controlar su salida.

1 — 2 — 3 — 4 — 5

Clase 1

APIs desde Python
latencia y tokens

Clase 2

Salida estructurada
Pydantic y tools

Clase 3

Contexto
y sesiones

Clase 4

FastAPI
streaming y errores

Clase 5

LLMOps
evaluación y despliegue

Hoy cambiamos la aproximación: ya sabemos llamar a un LLM; ahora necesitamos que su respuesta sea consumible por una aplicación.

Objetivos de aprendizaje de la sesión

Al final de la clase, el alumno debe poder convertir texto libre en un objeto de negocio validado y preparado para automatización.

1 Diseñar prompts operativos

Especificar tarea, reglas, formato, vocabularios y comportamiento ante incertidumbre.

2 Distinguir formatos estructurados

Entender JSON por prompt como baseline y sus límites como contrato técnico.

3 Validar con Pydantic

Convertir output probabilístico en un objeto tipado, validado y controlado.

4 Usar tool calls como schema

Forzar argumentos estructurados sin entrar aún en agentes ni ejecución de herramientas.

5 Usar Structured Outputs nativos

Pasar el modelo Pydantic como schema fuera del prompt mediante output_format.

6 Comparar enfoques

Elegir según robustez, portabilidad y coste de integración.

El cambio clave: de respuesta a objeto de negocio

Una automatización necesita datos que se puedan procesar, no solo texto que un humano pueda leer.

Respuesta libre

Útil para leer
Difícil de parsear

Prompt operativo

Define tarea, reglas y vocabulario
Reduce ambigüedad

Structured output

Objeto tipado
Errores controlados

Tool call + schema

Argumentos estructurados

Workflow seguro

Automatizar o escalar a revisión humana

Mayor control de formato, validación y comportamiento de la aplicación

Principio de ingeniería: no automatizar una salida que no puedes validar.

EBIS Business Techschool

BLOQUE 1

Prompting operativo

El prompt deja de ser una pregunta y se convierte en un contrato de automatización.

Una respuesta libre no es una automatización

Puede ser útil para un humano, pero frágil para un sistema que tiene que decidir o accionar.

free text

This looks like a refund request. The customer is unhappy because the item arrived broken. I would recommend checking the order and starting the refund process.

No parseable

No hay campos estables ni tipos definidos.

Ambigua

La prioridad y el proceso no están normalizados.

No testable

Cada ejecución puede cambiar de forma.

No accionable

No puede alimentar directamente un workflow.

Respuesta libre	Salida estructurada
“It looks like a refund request…”	`"process_type": "refund_request"`
“The customer is unhappy…”	`"priority": "medium"`
“Order may need review…”	`"order_id": "XXXX"`
“Start the refund process…”	`"suggested_action": "Validate refund eligibility"`

En automatización, el prompt es un contrato

Debe describir qué se espera, qué valores están permitidos y qué hacer cuando hay incertidumbre.

1 Rol

Qué perspectiva adopta el modelo

2 Tarea

Qué debe resolver exactamente

3 Datos

Dónde empieza y termina el input

4 Reglas

Criterios de negocio y seguridad

5 Formato

Cómo debe devolver el resultado

6 Fallback

Qué hacer si falta información

La diferencia entre “preguntar” y “automatizar” es la existencia de un contrato verificable.

Mismo caso, dos prompts muy diferentes

La calidad del output cambia radicalmente cuando el prompt define contrato y vocabulario.

Prompt débil

```
Analyze this customer request and tell me what to do.
```

Prompt operativo

```
You are a customer service automation assistant.

Classify the request using the allowed process types.
Extract the order ID only if it is explicitly present.
Do not invent missing information.
Set requires_human_review to true when the request
is ambiguous, incomplete or high-risk.
```

Dimensión	Prompt débil	Prompt operativo
Tarea	Abierta	Clasificar y extraer
Categorías	No definidas	Vocabulario controlado
Campos	Implícitos	Reglas explícitas
Datos ausentes	Puede inferir	Usa `null`
Incertidumbre	No gestionada	Revisión humana
Formato técnico	No controlado	Schema separado en la API

Anatomía de un prompt operativo

La plantilla debe separar instrucciones, datos y salida esperada.

template

```
ROLE
You are a customer service automation assistant.

TASK
Classify the request and extract the information
required by the customer service workflow.

INPUT
<request>
{customer_request}
</request>

RULES
- Do not invent missing values
- Use null when information is absent
- Set requires_human_review=true when ambiguous
- Keep suggested_action short and operational
```

Role

Alinea tono y responsabilidad del modelo.

Task

Reduce ambigüedad: clasificar, extraer, normalizar.

Input

Delimita los datos para evitar mezclar instrucciones y texto.

Rules

Fija criterios de negocio, seguridad e incertidumbre.

Schema — fuera del prompt

Pydantic define campos, tipos, enums y restricciones mediante output_format.

No duplicamos el schema en el prompt: el prompt explica el negocio y la API controla la estructura.

Los vocabularios controlados reducen ambigüedad

El modelo puede seguir siendo probabilístico, pero el dominio de salida no tiene por qué ser libre.

Texto del usuario	Valor normalizado	Acción del sistema
“I want my money back”	`refund_request`	Abrir flujo de reembolso
“The item arrived broken”	`refund_request`	Solicitar validación de pedido
“Change my delivery address”	`delivery_change`	Enviar a logística
“I was charged twice”	`billing_issue`	Priorizar revisión de facturación
“I cannot access my account”	`technical_support`	Abrir ticket técnico

Regla práctica

Cuanto más cerrado sea el vocabulario, más fácil será validar y automatizar.

Riesgo

Si el catálogo no cubre casos reales, el modelo forzará clasificaciones incorrectas.

Diseño

Incluir “other” y revisión humana para gestionar incertidumbre.

Un LLM interpreta lenguaje flexible. El sistema consume etiquetas estables.

EBIS Business Techschool

BLOQUE 2

Salidas estructuradas

JSON ayuda, pero la aplicación debe validar y controlar el contrato.

El output objetivo: un objeto de negocio

La respuesta deja de ser un texto y pasa a ser una entidad que puede alimentar un flujo.

JSON object

```
{
  "process_type": "refund_request",
  "priority": "medium",
  "order_id": "8921",
  "reason": "damaged_item",
  "requires_human_review": true,
  "suggested_action": "Validate refund eligibility"
}
```

process_type

Selecciona workflow

priority

Ordena trabajo

order_id

Permite consultar sistema

reason

Normaliza causa

review

Controla seguridad

action

Siguiente paso

Objetivo: transformar input no estructurado en una pieza de software con campos esperados.

Una estructura válida no significa una decisión correcta

Hay varios niveles de control y cada uno resuelve un problema diferente.

Nivel	Pregunta	Control
Formato	¿La respuesta tiene la estructura esperada?	Structured Output de la API
Schema	¿Los campos, tipos y valores son válidos?	Pydantic
Semántica	¿La clasificación y extracción son correctas?	Dataset de evaluación y métricas
Negocio	¿Es seguro ejecutar la acción?	Reglas, umbrales y revisión humana

Una estructura válida puede fallar semánticamente; por eso combinamos schema, evaluación y reglas de negocio.

Tres niveles para obtener salidas estructuradas

El nivel 1 explica el problema; la práctica se centra en Structured Outputs y tool calls.

1 Nivel 1 — Prompt JSON

El modelo sigue una instrucción textual, pero la API no garantiza la estructura.

Ventaja: simple y portable.
Riesgo: texto extra, markdown, campos incorrectos o JSON inválido.

Uso: prototipos y bajo riesgo; siempre con validación posterior.

2 Nivel 2 — Structured Output nativo

El schema se pasa como configuración técnica de la API, separado del prompt.

Ventaja: mayor control del formato.
Riesgo: estructura válida no garantiza decisión correcta.

Uso: extracción, clasificación y normalización de objetos de negocio.

3 Nivel 3 — Tool call

El contrato se define como argumentos de una herramienta. Claude devuelve un bloque tool_use con un objeto estructurado.

Ventaja: prepara flujos con herramientas.
Riesgo: más complejidad e integración.

Uso: cuando el output representa una operación ejecutable.

Structured Output devuelve un objeto; tool call solicita una operación con argumentos.

Nivel 1 — Prompt-only JSON

El enfoque más simple: pedir formato estructurado directamente en el prompt.

prompt

```
Analyze the customer request and return only valid JSON.

JSON schema:
{
  "process_type": "...",
  "priority": "...",
  "order_id": "... or null",
  "requires_human_review": true/false
}

Rules:
- Return only JSON
- Do not include markdown
- Use null for missing fields
```

Ventaja

Muy simple y portable entre proveedores.

Riesgo

Puede devolver texto extra, markdown o campos inválidos.

Uso recomendado

Prototipos y casos de bajo riesgo; siempre con validación posterior.

Errores típicos de salida estructurada

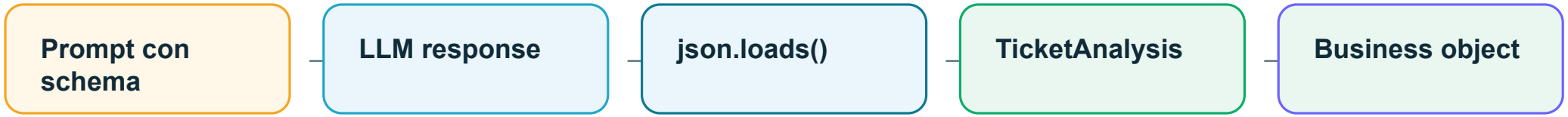
El modelo puede cumplir la intención, pero romper el contrato que necesita el software.

Error	Qué significa	Ejemplo
JSON inválido	No se puede parsear	Texto antes del `{}`
Falta campo obligatorio	El contrato está incompleto	No viene `priority`
Valor no permitido	Rompe vocabulario cerrado	`priority = urgent`
Dato inventado	El modelo alucina un campo	`order_id` no presente
Salida demasiado larga	No es operativa	Explicación dentro del JSON

La salida inválida no es un fallo menor: es una señal para no automatizar sin control.

Caso 1 — Prompt + JSON + Pydantic

El contrato no termina en el prompt: termina cuando la aplicación valida el objeto.



Qué hace Pydantic

Valida campos, tipos, enums, opcionales y restricciones.

Qué no hace Pydantic

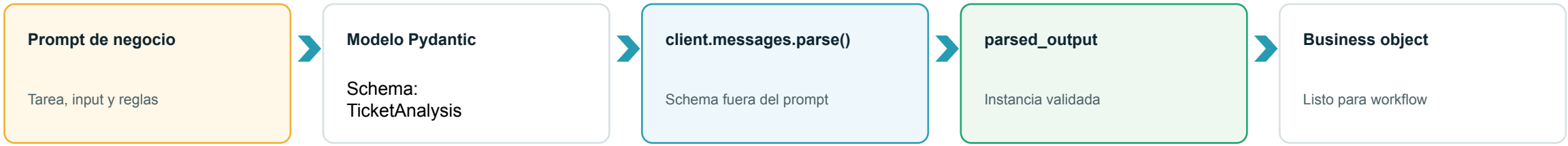
No garantiza que la clasificación sea correcta.

Validación

Pydantic valida después de recibir la respuesta.

Nivel 2 — Structured Output nativo

El schema se pasa como configuración técnica de la API, separado del prompt.



El schema no se escribe en el prompt: se proporciona como configuración técnica de la llamada.

Schema: tipos, enums y campos opcionales

Definimos el contrato de salida una vez y lo reutilizamos en Structured Outputs y tool calls.

schemas.py

```python
from enum import Enum
from typing import Optional
from pydantic import BaseModel, Field

class ProcessType(str, Enum):
    refund_request = "refund_request"
    delivery_change = "delivery_change"
    delivery_delay = "delivery_delay"
    technical_support = "technical_support"
    billing_issue = "billing_issue"
    product_question = "product_question"
    account_management = "account_management"
    other = "other"

class Priority(str, Enum):
    low = "low"
    medium = "medium"
    high = "high"

class TicketAnalysis(BaseModel):
    process_type: ProcessType
    priority: Priority
    order_id: Optional[str] = None
    reason: Optional[str] = None
    requires_human_review: bool
    suggested_action: str = Field(min_length=3, max_length=180)
```

Qué aporta

Un contrato único para prompt, API, validación y CSV final.

Qué validamos

Tipos, campos requeridos, enums, opcionales y restricciones.

Qué no validamos aún

Que la clasificación sea correcta; eso requiere evaluación.

Prompt operativo sin duplicar el schema

El prompt describe la tarea; el contrato técnico va por output_format.

prompts.py

```
def build_business_prompt (customer_request: str, config: dict) -> str:
    return f"""
You are a customer service automation assistant.

Task:
Analyze the customer request and convert it into
an operational workflow object.

Customer request:
<request>
{customer_request}
</request>

Business rules:
- Do not invent missing information.
- Use null when a value is not explicitly present.
- Set requires_human_review to true when the request
  is ambiguous, incomplete or high-risk.
- Keep suggested_action short and operational.
""".strip()
```

Diseño

El prompt explica negocio, reglas y tratamiento de incertidumbre.

Separación

El schema no viaja como texto: se pasa en output_format.

Regla clave

No inventar datos; usar null y revisión humana cuando haga falta.

Structured Output: parsear y validar en una llamada

La API guía la forma de la respuesta y el SDK devuelve un objeto Pydantic.

structured_extract_2.py

```
try:
    response = client.messages.parse(
        model=model,
        max_tokens=config["llm"]["max_tokens"],
        temperature=config["llm"]["temperature"],
        messages=[{"role": "user", "content": prompt}],
        output_format=TicketAnalysis,
    )

    analysis = response.parsed_output
    status = "valid"
    error = ""

except (ValidationError, ValueError) as e:
    analysis = None
    status = "error"
    error = str(e)
```

Qué cambia

No hacemos json.loads(): el SDK entrega parsed_output.

Objeto final

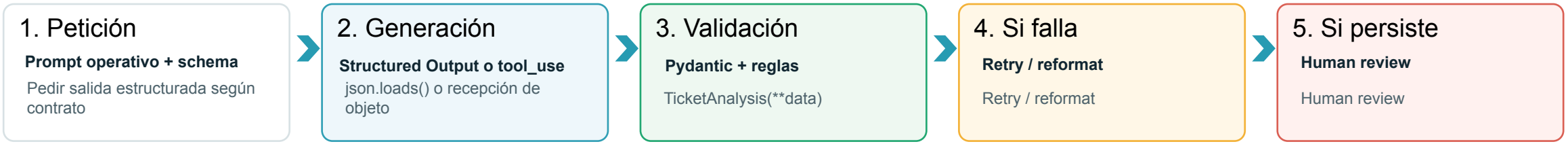
analysis ya es una instancia de TicketAnalysis.

Límite

Objeto válido no implica decisión correcta de negocio.

Respuesta segura ante errores

La robustez no consiste en ocultar errores, sino en tener una salida segura cuando ocurren.



Automatización responsable: ante incertidumbre, escalar. No inventar ni forzar una acción.

EBIS Business Techschool

BLOQUE 3

Tool calls como schema

No hacemos agentes: usamos la tool definition como mecanismo de salida estructurada.

Tool calls no significa necesariamente agentes

En esta clase usamos tools solo para obtener argumentos estructurados, sin ejecutar acciones externas.

Uso en esta clase	Uso en agentes
Una llamada puntual	Loop multi-paso
Extraer argumentos estructurados	Decidir acciones y herramientas
No ejecutamos efectos externos	Puede ejecutar APIs o procesos
Se valida con Pydantic	Se observa y controla el ciclo

Tool calls como salida estructurada. Los agentes, tools con efectos y loops se verán más adelante.

Tool schema: el contrato se mueve a la API

La herramienta define nombre, descripción y un input_schema con los campos esperados.

tool_schema.py

```
TICKET_ANALYSIS_TOOL = {
  "name": "extract_ticket_analysis",
  "description": "Extract structured information from a customer request.",
  "input_schema": {
    "type": "object",
    "properties": {
      "process_type": {
        "type": "string",
        "enum": [item.value for item in ProcessType]
      },
      "priority": {
        "type": "string",
        "enum": [item.value for item in Priority]
      },
      "order_id": {"type": ["string", "null"]},
      "reason": {"type": ["string", "null"]},
      "requires_human_review": {"type": "boolean"},
      "suggested_action": {"type": "string"}
    },
    "required": ["process_type", "priority", "order_id", "reason",
                 "requires_human_review", "suggested_action"]
  }
}
```

Qué cambia

El schema se declara como input_schema de una herramienta, no como respuesta directa.

Qué controla

Nombre de la tool, campos, tipos, enums, requeridos y descripciones.

Qué sigue faltando

Validación de aplicación, reglas de negocio y revisión humana.

Forzar el tool call en una llamada

El modelo devuelve un bloque tool_use en lugar de una respuesta narrativa.

tool_extract.py

```
response = client.messages.create(
    model=model,
    max_tokens=config["llm"]["max_tokens"],
    temperature=0.0,
    tools=[TICKET_ANALYSIS_TOOL],
    tool_choice={
        "type": "tool",
        "name": "extract_ticket_analysis",
    },
    messages=[{"role": "user", "content": prompt}],
)
```

tool_choice

Fuerza una herramienta concreta para ese request.

temperature=0.0

Reducimos variación en tareas de extracción y clasificación.

Aun así

Aunque haya tool_use, seguimos validando con Pydantic.

Extraer el bloque tool_use y validarlo

La tool call devuelve argumentos estructurados; la aplicación los valida igual que antes.

TODOs

```
tool_input = None

for block in response.content:
    if block.type == "tool_use" and block.name == "extract_ticket_analysis":
        tool_input = block.input
        break

if tool_input is None:
    raise ValueError("No tool_use block returned")

analysis = TicketAnalysis(**tool_input)
```

solución

```
analysis = TicketAnalysis(**tool_input)
status = "valid"
```

Best practice: si no hay tool_use, no fuerces una acción. Registra error y escala.

Structured Output vs Tool call

La diferencia no es el schema: es si el modelo devuelve un objeto o solicita una operación.

Criterio	Structured Output + Pydantic	Tool call + Pydantic
Salida	Objeto final del modelo	Argumentos de una herramienta
Uso típico	Extraer, clasificar, normalizar	Preparar una acción o integración
Complejidad	Media-baja	Media-alta
Control de formato	Alto	Alto
Mensaje clave	Objeto validado	Operación con argumentos

EBIS Business Techschool

BLOQUE 4

Práctica guiada

Construimos dos pipelines de extracción y comparamos el resultado.

Carpeta de práctica de la Clase 2

Reutilizamos el cliente de la Clase 1 y añadimos schemas, prompts y dos mecanismos estructurados.

project

```
class_02_structured_outputs/
│
├── .env
├── llm_client.py
├── schemas.py
├── prompts.py
├── structured_extract.py        # baseline JSON prompt
├── structured_extract_2.py      # Structured Output
├── tool_schema.py
├── tool_extract.py
│
├── data/
│   └── customer_requests_structured.csv
│
└── outputs/
    ├── prompt_json_results.csv
    ├── structured_output_results.csv
    └── tool_call_results.csv
```

Reutilizamos de Clase 1

Cliente Anthropic, .env, config.yml y CSV de solicitudes.

Nuevo en Clase 2

schemas.py, prompts.py y structured_extract_2.py con messages.parse().

Extra avanzado

tool_schema.py y tool_extract.py para extracción mediante tool_use.

Práctica A — Structured Output nativo + Pydantic

El alumno completa el schema y genera un CSV de objetos estructurados.

1. Completar Priority enum

schemas.py

2. Añadir campos opcionales

order_id y reason

3. Definir reglas del prompt

prompts.py

4. Llamar a client.messages.parse

structured_extract_2.py

5. Leer response.parsed_output

TicketAnalysis

6. Guardar structured_output_results.csv

outputs/

Objetivo: obtener objetos válidos sin duplicar el schema en el prompt y registrar errores controlados.

Práctica B — Tool call como extracción estructurada

El alumno compara el objeto directo de Structured Outputs con los argumentos devueltos por tool_use.

TODOs

```
# TODOs in tool_extract.py

tool_input = None

for block in response.content:
    # identify the tool_use block
    # extract block.input
    pass

if tool_input is None:
    raise ValueError("No tool_use block")

analysis = TicketAnalysis(**tool_input)
```

Resultado esperado

tool_call_results.csv con los mismos campos de negocio que Structured Outputs.

Comparativa

¿Cambia la latencia, los tokens o la estabilidad del formato?

Mensaje clave

No es un agente: no ejecutamos nada. Solo capturamos argumentos estructurados.

Qué tiene que aparecer al final

El resultado debe ser un CSV con objetos estructurados y trazabilidad básica.

request_id	status	process_type	priority	review	latency_ms
REQ-001	valid	refund_request	medium	true	1240
REQ-002	valid	delivery_change	medium	false	1185
REQ-001	valid	refund_request	medium	true	1310
REQ-002	valid	delivery_change	medium	false	1265

Comparad método, validez, campos críticos, latencia y tokens antes de automatizar.

Mini-quiz de cierre

1. ¿Por qué no basta un output en texto libre?

2. ¿Qué garantiza Pydantic y qué no?

3. ¿Qué cambia al usar Structured Outputs?

4. ¿Cuándo usarías un tool call?

5. ¿Por qué validar tras recibir tool_use?

To sum up

convertir una respuesta LLM en un contrato de software validado.

1

1. Prompting operativo ≠ prompt bonito

La clave es diseñar un contrato que otro sistema pueda consumir.

2

El baseline es útil para entender el problema, pero no es un contrato robusto.

El output debe parsearse y validarse contra un schema explícito.

3

Tipos, enums y restricciones convierten la salida en objeto de aplicación.

La aplicación puede trabajar con tipos, enums y campos obligatorios.

4

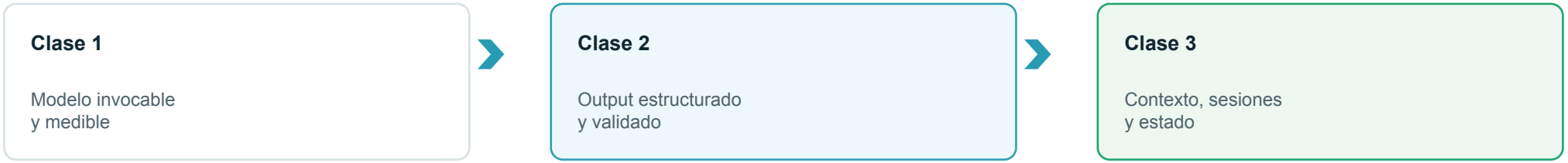
Structured Outputs devuelven objetos; tool calls preparan operaciones.

No son solo para agentes; también sirven como mecanismo de salida estructurada.

prompt, schema y validación deben trabajar juntos antes de automatizar.

de salida controlada a conversación con estado

La siguiente sesión añadirá contexto multi-turn sin entrar todavía en RAG.



Primero hacemos que la salida sea segura. Después haremos que el sistema recuerde y mantenga estado.

Referencias y documentación

Fuentes recomendadas para ampliar la parte técnica.

Pydantic documentation

Schemas, BaseModel, enums and validation errors.

Structured Outputs, messages.parse(), tool use, input_schema and tool_choice.

Tool use, input_schema, tool_choice and tool_use blocks.

Nota técnica

Structured Output devuelve un objeto; tool call devuelve argumentos de una herramienta.

Próximo paso

Conectar salida validada con contexto y memoria de sesión.